

CloudStore

A Beginner's Guide

What this project is, why we built it, and how it works explained in plain English with no jargon.

Document	Beginner's Guide
Date	May 16, 2026
For	Reviewers, teammates, anyone curious

1. What is this project?

CloudStore is a small website where people can store their personal files online. Think of it as a tiny version of Google Drive or Dropbox that we built ourselves.

Each person creates an account, logs in, and gets a private space where they can upload, download, and delete their own files. A separate **admin account** can see everyone and everything - useful for the person running the service.

2. Why did we build it?

This is a **Virtual Cloud Computing (VCC)** project. The goal is not just to write a website - it is to show that we understand how a real cloud service is put together end to end. To prove that, we did three things:

- **Built a real working web application** - not just slides or diagrams.
- **Deployed it on a virtual machine** - the same idea every cloud provider (AWS, Azure, Google Cloud) uses behind the scenes.
- **Made another computer talk to it over the network** - just like users would reach a real cloud service over the internet.

So this single project touches application development, databases, security, virtualization, and networking - all the building blocks of cloud computing.

3. The big picture

Here is what happens when you use the website. You click something in your browser, your browser sends that request to our app, and the app saves or fetches things from two places: a database (for information about files) and the disk (for the actual file content).

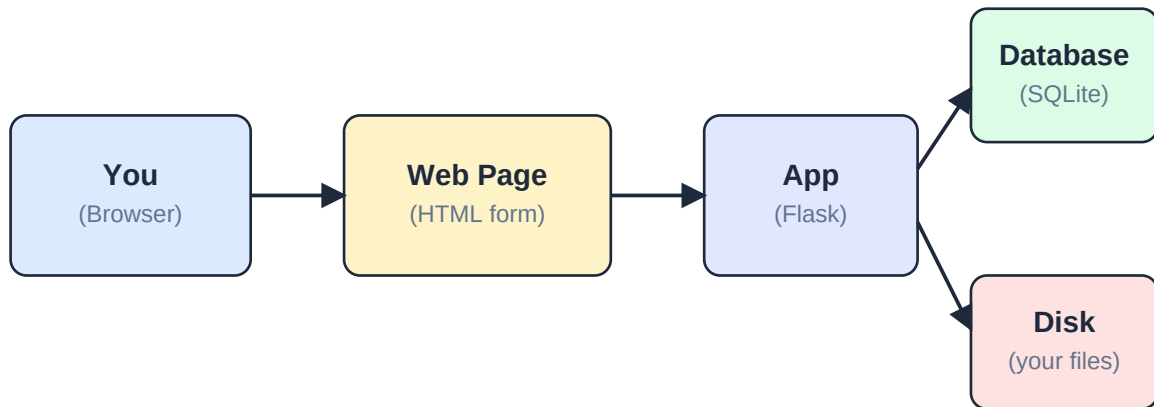


Figure 1 - The big picture

- **You** use a normal web browser - Chrome, Edge, anything.
- **The web page** shows you forms (login, upload, file list).
- **The app** is the brain. It checks who you are and decides what to do.
- **The database** remembers things like usernames, passwords (hashed), and file info.
- **The disk** holds the actual file bytes.

4. What can you actually do on the site?

There are two kinds of accounts. Regular users see only their own stuff. Admins see everything. The system enforces this on every single click - it is not just buttons being hidden, the server actively blocks unauthorised requests.

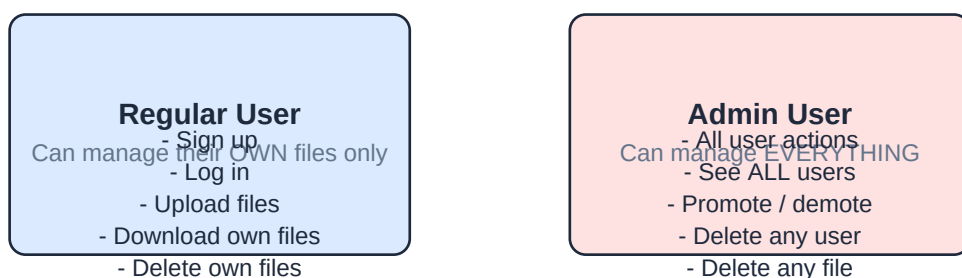


Figure 2 - The two types of accounts

5. A walk through the website

If you opened the site right now, this is the journey you would take.

Step 1 - Sign up

You visit the site and click **Signup**. You pick a username, an email, and a password. The website never stores your password as you typed it - it runs it through a one-way scrambler called **bcrypt** and only stores the scrambled version. Even we cannot read it back.

Step 2 - Log in

You type your email and password. The site scrambles what you typed and compares it to what is stored. If they match, you are in. The site puts a small **session cookie** in your browser so it remembers you on the next click.

Step 3 - Upload a file

You land on your dashboard. You click **Choose File**, pick something from your computer, and hit **Upload**. Two safety things happen before the file lands on disk:

- The filename is **cleaned** - dangerous characters like slashes or dots are removed so nobody can use a sneaky filename to escape into other folders.
- A **random identifier** is added to the front of the name so two people uploading *report.pdf* never overwrite each other.

Step 4 - Download a file

You click **Download** next to a file in your list. Before sending you anything, the app checks one critical thing: **does this file actually belong to you?** If not, it refuses with a 403 error. Admins are the only exception - they can download any file.

Step 5 - Delete a file

Same protection. You can delete your own files. Admins can delete anyone's. The app removes the file from disk *and* the entry from the database in one go.

6. How we built it (our process)

We worked in five phases. Each phase had a clear goal so we could check progress as we went.

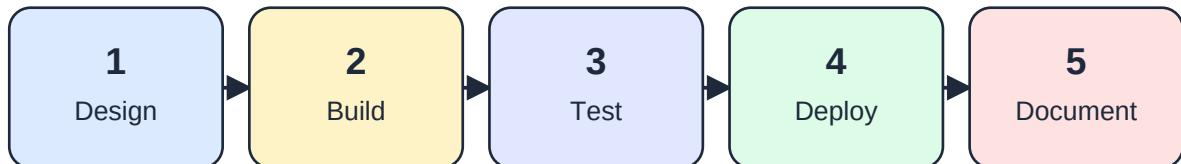


Figure 3 - Our five project phases

Phase	What we did	Why it mattered
1. Design	Wrote down the requirements: who uses it, stops the casting what building server.	Stop the casting what building server.
2. Build	Coded the Flask app, database tables, and HTML pages.	THE ACTUAL PRODUCT.
3. Test	Ran the app on Windows, signed up, uploaded, bugs, fixed the dead inputs.	Get bugs fixed, the dead inputs.
4. Deploy	Created an Ubuntu virtual machine, transferred the project to the server.	Proves the project to the server.
5. Document	Wrote the README and three PDFs (this one, the design spec, and the full guide).	So anyone can pick it up and build it.

7. Why use a virtual machine?

A virtual machine (VM) is a complete computer pretending to live inside your real computer. We made a VM running **Ubuntu Server 22.04** inside VirtualBox. The app runs there, not on Windows.

Why bother? Three reasons:

- **It looks like a real server.** Real cloud servers run Linux. By using a Linux VM we deal with the same filesystem, same package manager, same firewall rules as a production system.
- **It is isolated.** Whatever we do inside the VM cannot break Windows. We can delete it and start over without losing anything.
- **It demonstrates networking.** Two separate machines (Windows host and Linux VM) talking to each other over a LAN is exactly how clients reach real cloud services.

8. The tools we used (in plain English)

Tool	What it is	What it does for us
Python	A programming language	The language the whole app is written in.
Flask	A small web framework	Handles requests and responses.
SQLite	A simple file-based database	Stores users, file info, categories.
bcrypt	A password hashing tool	Scrambles passwords so they can never be read back.
Jinja2	A template engine	Fills HTML pages with live data.
VirtualBox	Virtual machine software	Runs the Ubuntu VM inside Windows.
Ubuntu Server	A Linux operating system	What the VM runs - our 'cloud server'.
SSH / SCP	Secure remote tools	Used to copy files from Windows to the VM.

9. What the website looks like

The site is intentionally simple - clean forms, a navigation bar at the top, and a single stylesheet. Here are the main pages:

Page	Purpose
Login	Type email and password to enter.
Signup	Create a new account.
My files (Dashboard)	See your own files in a table. Upload form sits at the top.
Admin dashboard	Counts of users, files, and total storage used.
Admin > Users	List of all users with promote / delete buttons.
Admin > All files	List of every file in the system regardless of owner.
Admin > Categories	Add labels like 'Documents', 'Images' that users can apply at upload.

10. What about security?

Security is built in at every level. The short version:

- Passwords are **hashed with bcrypt** - never stored in plain text.
- Every form has a **CSRF token** - stops other websites tricking your browser.

- **Filenames are sanitised** - no tricks with slashes or dots to escape folders.
- **Every download and delete** checks the file belongs to you first.
- **Admin pages** are blocked at the server, not just hidden in the UI.
- **Secrets** (like the session key) live in a .env file, not in the source code.

11. In one paragraph

We built a small but real cloud file storage website. People sign up, log in, and upload files into their own private space. Admins manage everyone. We wrote it in Python with Flask, stored data in a SQLite database, then deployed the whole thing onto an Ubuntu Server virtual machine running in VirtualBox. The Windows host opens the site over the local network - exactly the way users reach real cloud services over the internet. The result is a working demonstration of the core ideas behind cloud computing: building, deploying, securing, and operating a service that lives on a server somewhere else.

Where to learn more

- **CloudStore_Documentation.pdf** - the concise design specification.
- **CloudStore_Full_Guide.pdf** - the full operator and developer guide with diagrams for login, upload, download, and architecture.
- **README.md** - quick setup commands for Windows and Linux.

Generated by docs/generate_basic_pdf.py. Re-run that script whenever the project changes.